Migration Guide: ReportViewerCore.WinForms v15.0

Overview

Version 15.x introduces breaking changes to improve resource management and prevent memory leaks. This guide helps you migrate from v15.0 to v15.0.

Breaking Changes

PrintDialog Resource Management

What Changed

In v15.0 and earlier, the `CustomPrintDialog.GetPrintDialog()` method returned a `PrintDialog` object that was never disposed, causing resource leaks on every call.

In v15.0:

- **New Method**: `GetPrintDialogSettings()` returns a `PrintDialogSettings` object (no disposal needed)
- **Deprecated Method**: `GetPrintDialog()` is marked as `[Obsolete]` with a warning

Migration Steps

Option 1: Use GetPrintDialogSettings() (Recommended)

```
// OLD CODE (v15.0) - Resource leak!
var customDialog = new CustomPrintDialog(printerSettings, pageSettings);
var printDialog = customDialog.GetPrintDialog();
// PrintDialog was never disposed, causing memory leak
```

```
// NEW CODE (v15.0) - No resource leak
var customDialog = new CustomPrintDialog(printerSettings, pageSettings);
var settings = customDialog.GetPrintDialogSettings();

// If you need a PrintDialog, create it and dispose properly:
using (var printDialog = settings.CreatePrintDialog())
{
    if (printDialog.ShowDialog() == DialogResult.OK)
    {
        // Use printDialog.PrinterSettings
    }
}
```

Option 2: Continue Using GetPrintDialog() with Proper Disposal

```
// v15.0 with explicit disposal
var customDialog = new CustomPrintDialog(printerSettings, pageSettings);
using (var printDialog = customDialog.GetPrintDialog())
{
    if (printDialog.ShowDialog() == DialogResult.OK)
    {
        // Use printDialog.PrinterSettings
    }
} // PrintDialog is properly disposed here
```

PrintDialogSettings API

The new `PrintDialogSettings` class provides all the configuration without holding resources:

```
public class PrintDialogSettings
{
    // Properties
    public string PrinterName { get; set; }
    public bool AllowSomePages { get; set; }
    public bool AllowSelection { get; set; }
    public bool AllowPrintToFile { get; set; }
    public bool PrintToFile { get; set; }
    public bool UseEXDialog { get; set; }
    public bool ShowNetwork { get; set; }
    public PrintRange PrintRange { get; set; }
    public short Copies { get; set; }
    public bool Collate { get; set; }
    public int FromPage { get; set; }
    public int ToPage { get; set; }
    public int MinimumPage { get; set; }
    public int MaximumPage { get; set; }

    // Methods
    public PrintDialog CreatePrintDialog(); // Caller must dispose
    public void ApplyTo(PrintDialog printDialog);
}
```

Usage Examples

Example 1: Simple Settings Access

```
var customDialog = new CustomPrintDialog(printerSettings, pageSettings);
var settings = customDialog.GetPrintDialogSettings();

// Access settings without creating PrintDialog
Console.WriteLine($"Printer: {settings.PrinterName}");
```

```
Console.WriteLine($"Copies: {settings.Copies}");
Console.WriteLine($"Collate: {settings.Collate}");
```

Example 2: Creating PrintDialog When Needed

```
var customDialog = new CustomPrintDialog(printerSettings, pageSettings);
var settings = customDialog.GetPrintDialogSettings();

// Only create PrintDialog when showing UI
using (var printDialog = settings.CreatePrintDialog())
{
    if (printDialog.ShowDialog() == DialogResult.OK)
    {
        // User confirmed, use the settings
        PrintDocument(printDialog.PrinterSettings);
    }
}
```

Example 3: Applying Settings to Existing PrintDialog

```
var customDialog = new CustomPrintDialog(printerSettings, pageSettings);
var settings = customDialog.GetPrintDialogSettings();

using (var printDialog = new PrintDialog())
{
    settings.ApplyTo(printDialog);

    if (printDialog.ShowDialog() == DialogResult.OK)
    {
        // Use configured PrintDialog
    }
}
```

Additional Improvements in v15.0

Better Exception Diagnostics

Silent exception handlers now include debug logging to help identify issues:

```
// Previously: Silent failures
catch { return defaultValue; }

// Now: Logged failures
catch (Exception ex)
{
    Debug.WriteLine($"MethodName: {ex.GetType().Name} - {ex.Message}");
```

```
    Debug.WriteLine(ex.StackTrace);
    return defaultValue;
}
```

Improved Null Safety

Constructors now validate parameters to prevent NullReferenceException:

```
// v15.0 throws ArgumentNullException early
var customDialog = new CustomPrintDialog(null, pageSettings);
// Throws ArgumentNullException with clear message
```

Thread Cancellation Support

Improved cancellation support for report rendering operations (see ProcessingThread improvements).

Async/Await Best Practices

Library async methods now use `ConfigureAwait(false)` to prevent deadlocks in certain synchronization contexts.

Deprecation Timeline

- **v15.0**: `GetPrintDialog()` is marked with `[Obsolete(false)]` - generates compiler warning
- **v17.x and later**: Method will remain available for backward compatibility
- **Recommendation**: Migrate to `GetPrintDialogSettings()` at your earliest convenience

Version Numbering Change

Starting with v15.0, versions follow a date-based pattern:

- **Format**: `16.{yy.MMdd.HH}` (Release builds)
- **Format**: `16.{yy.MMdd}` (Debug builds)
- **Example**: `16.25.1231.14` = Version 16, built on Dec 31, 2025 at 2 PM

Need Help?

- Review the CHANGELOG.md for all changes
- Check the examples in the DOCS folder
- Report issues on GitHub: https://github.com/lkosson/reportviewercore/issues

Summary

The primary migration task is replacing `GetPrintDialog()` calls with `GetPrintDialogSettings()` or adding proper disposal. This change prevents resource leaks and improves application stability. The migration is straightforward and can be done incrementally, as the old method remains available with a warning.